МИНОБРНАУКИ РОССИИ

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ

ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра автоматики и процессов управления

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Программирование»

Тема: Вычислить и вывести на экран в виде таблицы значения функции F

Студент гр. 6391 Сопов Н.В.

Преподаватель Писарев А.С.

Санкт-Петербург

2026

Цель работы (задание)

Вычислить и вывести на экран в виде таблицы значения функции F на интервале от Хнач. до Хкон. с шагом dX.

$$F = \begin{cases} ax^2 + b, & \text{при } x < 0 \text{ и } b \neq 0 \\ \dfrac{x - a}{x - c}, & \text{при } x > 0 \text{ и } b = 0 \\ \dfrac{x}{c}, & \text{в остальных случаях} \end{cases}$$

где a, b, c – действительные числа.

Функция F должна принимать действительное значение, если выражение ($A_Ц$ ИЛИ $B_Ц$) И ($A_Ц$ ИЛИ $C_Ц$) не равно нулю, и целое значение - в противном случае. Через $A_Ц$, $B_Ц$, $C_Ц$ обозначены целые части значений a, b, c; операции И и ИЛИ – поразрядные. Значения a, b, c, $X_{нач.}$, $X_{кон.}$, dX ввести с клавиатуры.

Алгоритм решения

1. **Ввод данных.** С клавиатуры вводятся коэффициенты a, b, c и параметры интервала: Хнач., Хкон., dX.
2. **Проверка корректности ввода.** Если dX = 0 – выводится ошибка и программа завершается (иначе цикл зациклится).
3. **Вычисление целых частей коэффициентов.** Ac = int(a), Bc = int(b), Cc = int(c). Они нужны для проверки условия вывода: ((Ac | Bc) & (Ac | Cc)) != 0.
4. **Обход интервала.** Используется цикл for (double x = Xnach; x <= Xkon + (dX / 2.0); x += dX). Добавление (dX / 2.0) к Xkon позволяет компенсировать погрешности округления вещественных чисел и гарантировать включение последней точки в таблицу.
5. **Вычисление F.** Для каждой точки x проверяются условия выбора ветвей функции:
   - если и : ;
   - если и : проверяется знаменатель . Если он равен нулю, точка пропускается (оператор continue), иначе ;
   - в остальных случаях: проверяется знаменатель . Если , точка пропускается, иначе .
6. **Вывод значения:**
   - если ((Ac | Bc) & (Ac | Cc)) != 0 – выводится действительное значение F;
   - иначе – выводится целая часть F (отбрасывание дробной части через (int)F).

Текст программы

```
#include <iostream>
#include <cmath>   //для математических функций

using namespace std;

int main() {
    double a, b, c;
    double Xnach, Xkon, dX;
    cout << "Enter a, b, c: ";
    cin >> a >> b >> c;
    cout << "Enter Xnach, Xkon, dX: ";
    cin >> Xnach >> Xkon >> dX;

    if (dX == 0) {
        cout << "Error" << endl;//проверка на корректность шага
        return 1;
    }
    int Ac = (int)a;//избавление от дробей
    int Bc = (int)b;
    int Cc = (int)c;

    for (double x = Xnach; x <= Xkon + (dX / 2.0); x += dX) { //for для защиты от ошибок округления тк числа дробные
        double F;

        //первое усл
        if (x < 0 && b != 0) {
            F = a * x * x + b;
        }
        //второе усл
        else if (x > 0 && b == 0) {
            if (x - c != 0) { //проверка деления на ноль для (x - c)
                F = (x - a) / (x - c);
            } else {
```

```
            continue;
        }
    }
    //третье усл
    else {
        if (c != 0) { //проверка деления на ноль для c
            F = x / c;
        } else {
            continue;
        }
    }
    if (((Ac | Bc) & (Ac | Cc)) != 0) {
        cout << x << " " << F << endl;
    } else {
        int FF = (int)F;//избавляемся от дробей
        cout << x << " " << FF << endl;
    }
  }

  return 0;
}
```

Описание тестов

Для каждого теста указаны входные данные и ожидаемый результат.

Тест 1. Обычный случай – все три ветки функции

Вход: a = 2, b = 1, c = 3; Хнач. = −2, Хкон. = 2, dX = 1

Условие: ((2|1) & (2|3)) = 3 ≠ 0 → F выводится как действительное число.

Ожидаемый вывод:

-2 9

-1 3

0 0

1 0.333333

2 0.666667

Тест 2. Обычный случай – ветка (x−a)/(x−c) при b = 0

Вход: a = 1, b = 0, c = 4; Хнач. = 1, Хкон. = 5, dX = 1

Условие: ((1|0) & (1|4)) = 1 ≠ 0 → действительный

формат. Ожидаемый вывод:

1 -0

2 -0.5

3 -2

5 4

Тест 3. Граничный случай – деление на ноль (c = 0)

Вход: a = 1, b = 2, c = 0; Хнач. = −1, Хкон. = 1, dX = 1

Условие: ((1|2) & (1|0)) = 1 ≠ 0 → действительный формат (где определено).

Ожидаемый вывод:

-1 3

Тест 4. Граничный случай – условие вывода целых значений

Вход: a = 0, b = 0, c = 5; Xнач. = −2, Xкон. = 2, dX = 1

Условие: ((0|0) & (0|5)) = 0 → F выводится как целое

число. Ожидаемый вывод:

-2 0

-1 0

0 0

1 0

2 0

Тест 5 Граничный случай – Xнач. = Xкон. (одна точка)

Вход: a = 2, b = 3, c = 1 Xнач. = 5, Xкон. = 5, dX = 1

При Xнач. = Xкон. цикл for выполняется ровно один раз: x = 5, условие 5 <= 5 истинно, после чего шаг даёт 6, и цикл завершается.

Ожидаемый вывод:

5 5

Результат работы

Ниже приведены скриншоты консоли, подтверждающие корректную работу программы на разработанных тестах

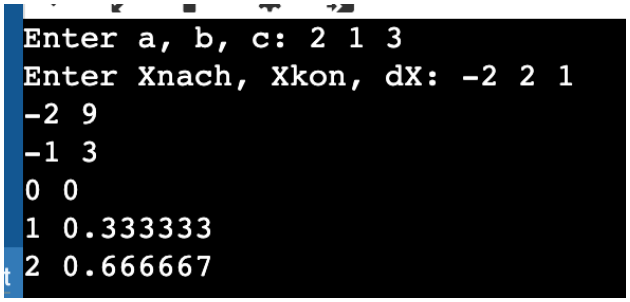


Рисунок 1. Результаты выполнения теста 1

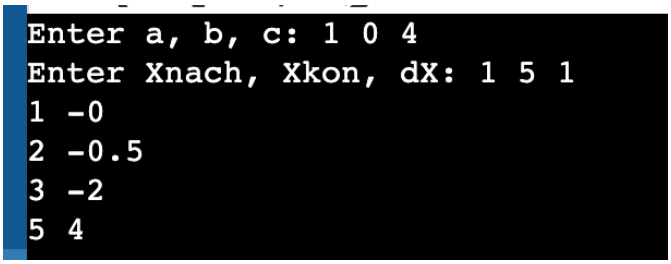


Рисунок 2. Результаты выполнения теста 2

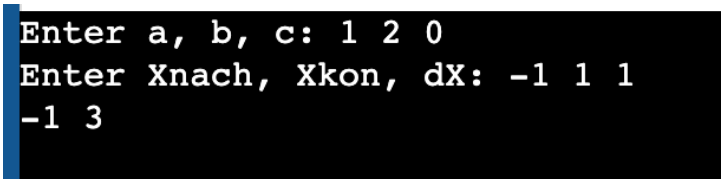


Рисунок 3. Результаты выполнения теста 3

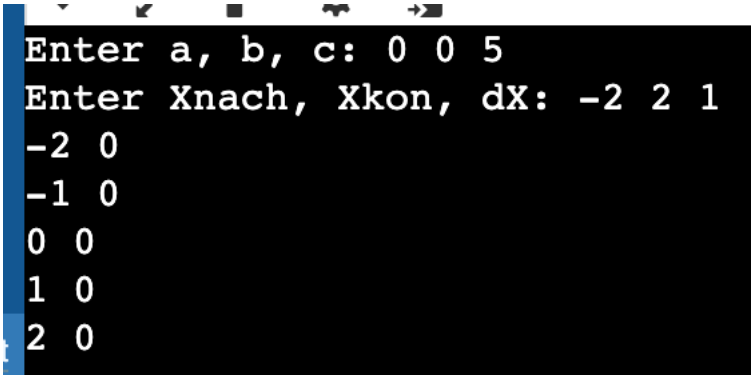


Рисунок 4. Результаты выполнения теста 4

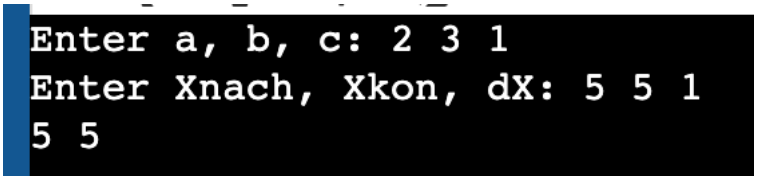


Рисунок 5. Результаты выполнения теста

Вывод

В ходе лабораторной работы была разработана программа на языке **C++**, вычисляющая значения кусочно-заданной функции F на заданном интервале с заданным шагом.

Для корректной работы с любым знаком шага обход интервала реализован через конструкцию if / else с двумя циклами for (double x = Xn; ...; x += dX): при положительном шаге используется условие x <= Xk, при отрицательном – x >= Xk.

Логика вычисления F вынесена в отдельную функцию, что позволило избежать дублирования кода между двумя циклами.

Учтены возможные особые случаи: деление на ноль (точка помечается как undefined), некорректный ввод (нулевой шаг, несовпадение направления шага и интервала).

Условие выбора между действительным и целым выводом реализовано через поразрядные операции И и ИЛИ над целыми частями коэффициентов, как указано в задании.

Тестирование подтвердило правильность работы программы на различных наборах данных.

В процессе работы закреплены навыки работы с условными операторами, циклами, вещественной арифметикой, функциями и поразрядными операциями в языке C++.